«usa» lifespan: loop_reentrenamiento_periodico()

«module»
main (entrypoint ASGI)

- _ROOT_ENV, _API_ENV: Path
+ MAX_EVENTO_BODY_BYTES = 262144 (256 KB)
+ app: FastAPI = create_scoring_app()

- _avisar_modos_degradados(): None
+ lifespan(app: FastAPI) «asynccontextmanager»
load_artifacts(); create_task(loop_reentrenamiento_periodico)
salvo SCORING_DISABLE_SCHEDULER=1
+ create_scoring_app(): FastAPI CORS, SCORING_DISABLE_DOCS
middleware limitar_tamano_body_eventos(request, call_next)
413 si POST /eventos > 256 KB

include_router

«router»
routes (APIRouter, header X-API-Key)

+ router: APIRouter
+ SIN_REPORTAR_SEGUNDOS = 900
+ DETALLE_CONFIRMACION_MAX = 200

+ POST /eventos: EventoIn → EventoScoreOut [agent] autorizar_agente + dedup
+ POST /usuarios/{uid}/confirmar-accion: ConfirmarAccionIn → ConfirmarAccionOut [agent]
+ GET /usuarios/{uid}/score → EventoScoreOut [api]
+ POST /usuarios/{uid}/reset → ResetOut [admin]
+ DELETE /usuarios/{uid} → 204 [admin]
+ POST /admin/reload-artifacts → ReloadArtifactsOut [admin]
+ GET /usuarios/{uid}/eventos-recientes → EventosRecientesOut [api]
+ GET /usuarios/{uid}/historial?desde&hasta&limit → HistorialScoresOut [api]
+ GET /usuarios/historial?limit → UsuariosHistorialOut [api]
+ GET /usuarios?limit&offset → UsuariosListOut [api]
+ GET | PUT /config/umbrales: UmbralesPutIn → UmbralesConfig [api | admin]
+ GET | PUT /config/acciones: AccionesConfig [api | admin]
+ GET | PUT /config/reentrenamiento: ReentrenamientoConfig [api | admin]
+ GET | POST /config/reentrenamiento-log: ReentrenamientoLogEntryIn/ListOut [api | admin]
+ GET /config/drift-alerts → DriftAlertListOut [api]
+ GET | PUT /config/eventos: EventosConfig [api | admin]
+ GET | PUT /config/aprendizaje: AprendizajeConfig [api | admin]
+ GET | PUT /config/iam: IAMConfig [api | admin]
+ POST /config/iam/probar → IAMProbarOut [admin]
+ GET | PUT /config/notificaciones: NotificacionesConfig [api | admin]
+ GET /config/auditoria → AuditoriaListOut [api]
+ GET /health → HealthStatus [sin auth]
+ GET /health/ready → ReadinessStatus (503 si no listo) [sin auth]
+ GET /health/metrics → HealthMetrics [api]
+ POST /auth/login: LoginIn → LoginOut [sin auth]
+ POST /auth/logout → 204 [token]
- _con_fallback_postgres(fn, *args, default, **kwargs): tuple[Any, bool]
- _fecha_str(valor): str
- _actor_desde_key(x_api_key: str | None): str FNV-1a 32 bits
- _snapshot_politica(form: str): dict | None
- _auditar(form: str, before: dict | None, x_api_key: str | None): None
- _iam_config_out(): IAMConfig
- _chequear_artefactos_y_redis(): tuple[bool, dict[str, bool], bool]
- _es_sano(artefactos_cargados: bool, redis_ok: bool): bool

«module»
reentrenamiento_scheduler

+ INTERVALO_CHEQUEO_SEGUNDOS = 300
- _NOTEBOOKS_ORDEN: list[str] Fase1 ... Fase6 .ipynb
+ ARCHIVOS_RESPALDO: tuple[str, ...] (17 rutas)

- _dir_respaldos(): Path
+ respaldar_artefactos(): Path
+ restaurar_artefactos(respaldo): None
+ correr_reentrenamiento(actor: str, fecha: str): tuple[bool, bool] «async»
- _dentro_de_ventana(ahora: datetime, ventana_horaria: str | None): bool
- _a_naive_local(dt: datetime): datetime
- _periodo_vencido(ahora, ultima_fecha: datetime | None,
periodo_dias: int): bool
+ debe_reentrenar(ahora: datetime, periodo_dias: int,
ventana_horaria: str | None, ultima_fecha: datetime | None,
automatico: bool): bool
+ construir_comandos_pipeline(): list[list[str]]
+ ejecutar_pipeline_reentrenamiento(comandos = None)
: tuple[bool, str] «async» create_subprocess_exec
- _tick(): None «async»
+ loop_reentrenamiento_periodico(): None «async»

REPO_ROOT

«module»
paths

- _MARCADOR = scripts/_score_common.py
+ REPO_ROOT: Path env SCORING_REPO_ROOT

- _resolver_repo_root(): Path

comparar_y_promover()

«module»
metricas_service (gate RF-21)

+ BENCH_DIR, PUNTUAR: Path
+ CORTE_TEMPORAL = "2021-08-02", PRIMER_DIA_TEST = "2021-08-03"
+ PRIMARIOS = ("S2_k2", "S2_k5", "S2_k10", "S3")
+ NIVELES_ALTO = {"alto", "critico"}
+ MARGEN_AUC = 0.02, MARGEN_FPR = 0.01
+ MODELO = "benchmark_dev"

+ precondicion(candidato: Path): str | None
+ metricas_benchmark(tag: str): dict roc_auc_score
+ decidir(cand: dict, vig: dict): tuple[bool, str]
+ evaluar_gate(candidato: Path, vigente: Path): dict «async»
+ comparar_y_promover(reentrenamiento_id: int | None,
respaldo: Path): tuple[bool, str] «async»

get/set_*

write_audit_entry, get_historial, list_*

«module»
config_store

+ UMBRALES_REDIS_KEY = "scoring:config:umbrales"
+ ACCIONES_REDIS_KEY = "scoring:config:acciones"
+ ACCIONES_VALIDAS = (alertar, terminar_procesos, denegar_carpeta,
terminar_y_denegar, suspender_proceso, notificar_admin)
+ ACCIONES_DEFAULT = {bajo, medio, alto: alertar; critico: terminar_y_denegar}
+ REENTRENAMIENTO_REDIS_KEY; PERIODO_REENTRENAMIENTO_DEFAULT_DIAS = 30 (1..365)
+ VENTANA_HORARIA_RE, _EMAIL_RE: re.Pattern
+ EMAILS_ADMIN_REDIS_KEY, NOTIF_NIVELES_REDIS_KEY
+ NOTIF_NIVELES_DEFAULT = {bajo: F, medio: T, alto: F, critico: F}
+ EVENTOS_RECIENTES_REDIS_KEY; límite default 30 (1..200)
+ UMBRAL_APRENDIZAJE_REDIS_KEY; UMBRAL_APRENDIZAJE_DEFAULT = 20 (1..1000)
+ IAM_REDIS_KEY = "scoring:config:iam"; IAM_MAX_PROVEEDORES = 5
+ IAM_TIPOS = ("keycloak", "entra", "mock"); _IAM_CAMPOS
- _IAM_NOMBRE_RE, _IAM_SECRET_ENV_RE, _IAM_URL_RE, _IAM_TENANT_RE

+ get_umbrales(artifacts: Artifacts): dict[str, tuple[float, float]]
+ set_umbrales(umbrales, nivel_minimo_piso: str, umbrales_vigentes,
acciones: dict | None = None, notif_niveles: dict | None = None): None MSET
- _validate_regla_piso(umbrales, nivel_minimo: str): None
- _validate_umbrales(umbrales): None
- _get_dict_config(key: str, default: dict): dict
+ get_acciones(): dict[str, str]
+ set_acciones(acciones: dict[str, str], umbrales_actuales: dict): None
- _sobrantes_y_faltantes(por_nivel: dict, umbrales_actuales: dict): str
- _validate_acciones(acciones, umbrales_actuales): None
- _get_reentrenamiento_dict(): dict
+ get_periodo_reentrenamiento(): int
+ set_reentrenamiento(dias: int, automatico: bool, ventana: str | None): None
- _validate_periodo_reentrenamiento(dias: int): None
+ get_ventana_horaria(): str | None
- _validate_ventana_horaria(ventana: str): None
+ get_reentrenamiento_automatico(): bool
+ get_emails_admin(): list[str]
- _validate_emails_admin(emails: list[str]): None
+ get_notif_niveles(): dict[str, bool]
+ set_notificaciones(emails: list[str], niveles: dict[str, bool],
umbrales_actuales: dict): None MSET
- _validate_notif_niveles(niveles, umbrales_actuales): None
+ get_eventos_recientes_limite(): int
+ set_eventos_recientes_limite(limite: int): None
- _validate_eventos_recientes_limite(limite: int): None
+ get_umbral_aprendizaje(): int
+ set_umbral_aprendizaje(umbral: int): None
- _validate_umbral_aprendizaje(umbral: int): None
+ get_iam_config(): list[dict]
+ set_iam_config(proveedores: list[dict]): None
- _validate_iam(proveedores: list[dict]): list[dict]

«module»
history_store (único acceso a Postgres)

+ SUPABASE_DB_URL: str | None env
- _engine: Engine | None «singleton» pool_size=2, max_overflow=3
- _errores_postgres_total: int

+ contar_error_postgres(): None
+ get_errores_postgres_total(): int
+ postgres_configurado(): bool
+ get_engine(): Engine
+ write_closed_day(uid, fecha, risk_score_hibrido, nivel_sugerido, cobertura,
accion_sugerida, explicacion = None, motivo = None): None UPSERT historial_scores
+ list_historial_uids(limit: int = 500): list[str]
+ delete_historial(uid: str): int
+ get_historial(uid, desde = None, hasta = None, limit = 90): list[dict]
+ write_audit_entry(form: str, actor: str | None, before: dict | None,
after: dict): None INSERT historial_politicas
+ list_audit_entries(limit: int = 50): list[dict]
+ write_reentrenamiento_log(fecha: str, cantidad_eventos: int,
variacion_metrica: str, actor: str | None): int | None historial_reentrenamientos
+ update_variacion_metrica(reentrenamiento_id: int, variacion_metrica: str): None
+ write_drift_alerts(uid: str, fecha: str, findings: list[DriftFinding]): None drift_alerts
+ list_drift_alerts(limit: int = 50): list[dict]
+ list_reentrenamiento_log(limit: int = 50): list[dict]
+ get_ultimo_reentrenamiento(): dict | None
+ contar_eventos_crudos(): int
+ write_evento_crudo(uid: str, ts: datetime, tipo_evento: str,
payload: dict): None INSERT eventos_crudos
+ write_metrica_modelo(modelo: str, metrica_valor: float | None,
cantidad_muestras: int | None, es_vigente: bool, rol = None,
metricas: dict | None = None, motivo = None): None metricas_modelo
+ marcar_metrica_no_vigente(modelo: str): None

write_metrica_modelo, update_variacion_metrica

escribe

Tablas Postgres (append-only, RLS sin políticas)

eventos_crudos	1 fila por evento (RF-19)
historial_scores	1 fila por uid y día cerrado
historial_politicas	auditoría de cada PUT /config/* (RF-17)
historial_reentrenamientos	log de reentrenamientos (RF-22)
drift_alerts	corrimiento del baseline personal
metricas_modelo	gate de promoción (RF-21)
historial_agent_keys	solo scripts/manage_agent_key.py

Postgres = historial; nunca se relee para decidir el score en vivo.

«usa»

«externa»
sqlalchemy.Engine

Dependencias en otras láminas
routes → hybrid_service, isoforest_service, features, explainability,
coverage, artifacts (B); auth, dedup_service, iam_service,
iam_providers (C); redis_state (A); schemas (E)
main → artifacts.load_artifacts (B), auth (C), redis_state (A)
config_store → artifacts (B), redis_state (A)
history_store → drift_service.DriftFinding (B)
reentrenamiento_scheduler → history_store, config_store (diferidos)